

Erklärung: projection_extension.dart

Projekt: Snackautomat (snackautomat_yakup_leandro)

Pfad: packages/power3d/lib/src/controller/projection_extension.dart

Umfang: ca. 3500 Zeilen

1. Was ist diese Datei?

Die Datei gehört zum Paket power3d und verbindet die Flutter-App mit der 3D-Szene (Babylon.js in einer WebView).

Sie sorgt dafür, dass Klicks, Kamera, Entnahmeklappe, Produktausgabe und Bestand im 3D-Automaten funktionieren.

Einfach gesagt: Flutter spricht Dart – die 3D-Welt spricht JavaScript. Diese Datei übersetzt dazwischen.

2. Warum ist sie so lang?

Die Datei besteht aus zwei Teilen:

Teil	Zeilen	Inhalt
A – Dart-API	1–315	Kurze Flutter-Methoden (Schnittstelle)
B – JavaScript	317–3564	Eingebetteter Script-Block _projectionHelpersJs für Babylon.js

Die Länge kommt nicht von 3500 Zeilen Flutter-UI, sondern weil die gesamte 3D-Automaten-Logik als JavaScript in derselben Datei steckt.

Flutter (Dart) → `_evalJs( . . . )` → Babylon.js (WebView)

kurze API lange 3D-Logik

3. Was macht der Dart-Teil?

Methode	Aufgabe
<code>projectHotspotQuads / pickAtScreen</code>	Positionen und Klicks auf dem 3D-Modell
<code>setDeliveryFlapOpen</code>	Entnahmeklappe öffnen / schließen
<code>dispenseItem</code>	Produkt ausgeben (Spirale → Aufzug → Fall)
<code>syncVendingStock</code>	sichtbare Produkte nach Bestand anpassen
<code>collectDispensedProduct</code>	ausgegebenes Produkt entnehmen
<code>ensureVendingPresentation</code>	Beleuchtung / Materialien
<code>updateVendingHudOled</code>	Inhalt auf dem Automaten-Display
<code>zoomAtScreen</code>	Zoom zur Mausposition

Flutter-Screens rufen diese Methoden auf; sie senden Befehle per JavaScript in die Szene.

4. Was macht der JavaScript-Teil?

Die eigentliche 3D-Arbeit in Babylon.js, zum Beispiel:

- Raycasting – Was trifft der Klick auf dem Modell?
- Klappen-Animation – Scharnier, Öffnungswinkel, Sichtbarkeit
- Aufzug / Korb – Fahrt zur Slot-Höhe und zurück
- Produkt-Meshes – ein-/ausblenden je nach Bestand
- Ausgabe-Sequenz – Spirale dreht, Produkt fällt, Aufzug parkt
- HUD / Materialien – Display-Textur und Look des Automaten

5. Merksatz für die Präsentation

„Die Datei wirkt riesig, weil Flutter und die 3D-Engine-Logik zusammenliegen. Der kurze Dart-Teil ist die API; der lange Teil ist Babylon-JavaScript.“

6. Vergleich im Projekt

Nur zwei Dart-Dateien im eigenen Code haben über 1000 Zeilen:

Datei	~Zeilen	Rolle
projection_extension.dart	~3500	3D-Brücke + Automaten-Logik
slide_materialize_content.dart	~1012	Inhalte der In-App-Präsentationsfolien

Weitere große Dateien liegen darunter (z. B. vending_machine_3d_landing_screen.dart ~700 Zeilen).

7. Wie man das als Team erklären kann

Ehrlich und verständlich:

„Die Datei gehört zu power3d. Sie verbindet Flutter mit der 3D-Szene. Der kurze Dart-Teil ist die Schnittstelle; der lange Teil ist JavaScript für Babylon.js (Animationen, Klicks, Automaten-Logik). Deshalb ist eine Datei so lang.“

Zur eigenen Rolle (je nach Beitrag anpassen):

- Anbindung der Methoden aus dem 3D-Screen (Klappe, Kauf, Bestand)
- Verständnis der Architektur Flutter ↔ WebView ↔ Babylon
- Gemeinsame Entwicklung / Bibliothek / Unterstützung – nicht Zeile für Zeile „von Hand aus dem Lehrbuch“